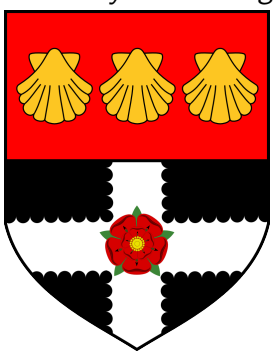
Numerical Simulation Methods And Their Applications

Monte Carlo Methods and Applications in Particle-based Systems

Ridhwaan Amin
32005096

Department of Mathematics and Statistics
University of Reading



May 15, 2026

Plain language summary

This project concerns working with a computer simulation of a particle-based system, particularly using numerical techniques known as Monte Carlo simulations.

It starts with an introduction to the history of these methods, touching upon the Manhattan Project, then focusing on pseudo-random number generators, sampling methodologies, and their applications in statistical mechanics. The project also explores the Metropolis method and its application to a Lennard-Jones liquid. Finally, it concludes with findings of a simulation created from individually developed C++ code.

These findings consist of the total energy within the system and the radial distribution function.

What we conclude is that the Monte Carlo method is highly effective at mimicking physics. By tracking their energy, we proved that simulated particles successfully settled into a stable liquid state. Measuring the distances between particles showed that they clustered together exactly as expected for a fluid. This demonstrates that these computational techniques are powerful tools for recreating and predicting the behaviour of complex materials without needing physical experiments, or increasingly large (and cost-heavy) models.

Declaration

I certify that this is my own work and that use of material from other sources has been properly and fully acknowledged in the text. I have read the University's definition of plagiarism and the department's advice on good academic practice. I understand that the consequence of committing plagiarism, if proven and in the absence of mitigating circumstances may include failure in the Year or Part or removal from the membership of the University. I also certify that neither this piece of work, nor any part of it, has been submitted in connection with another assessment.

In submitting this work I can confirm that I [~~have~~/have not] used Generative AI Tools to prepare and complete my assignment.

to prepare and complete work, please one select the following statements:

a. I acknowledge the use of [] to generate materials to support my research and understanding on the topic when drafting this assignment, but none of the outputs are included in the final version.

b. I acknowledge the use of [] to generate materials that were included within my final assessment in modified form. I have taken responsibility to ensure that where generated materials are included, these have been referenced appropriately, in accordance with the requirements of the assignment brief.

Contents

List of Tables

List of Figures

1 Development Of Computer Simulation Methods

The first concepts of Monte Carlo simulations can be found in the 18th century, contained within a report found in the 1733 edition of 'Histoire de L'Académie Royale des Sciences'. This report - created by Georges Louis Leclerc, Comte de Buffon - makes mention of an approximation of pi by throwing needles or rods atop a floor of boards, measuring the fractions that fell on a single board. Of course, this can be recreated in the modern day by a simulation of approximating pi using many points (see Appendices A and B). When initially proposed in this report, the field of Monte Carlo methods had not been distinctly named or partitioned (Landau & Binder 2021).

1.1 The Manhattan Project

1.1.1 Background Context

Before understanding Monte Carlo Methods we are bound to mention **The Manhattan Project**. The top-secret research and development program was the first to produce Nuclear Weapons, and was a key playing card in the context of the greater geopolitical climate: World War II.

Funded by nations such as the UK, Canada and, majorly, the USA, the Manhattan Project is often cited as a turning point in human history. One of the sites of the project was Los Alamos, New Mexico - tied intimately with the birth of Monte Carlo methods. Disregarding moral arguments concerning atomic weaponry, a large factor in the awe of the project was the pure advancement in scientific knowledgement - building upon previous discoveries of fission.

1.1.2 Stanisław Ulam

Stanisław Ulam was prominent in contributing to the founding of Monte Carlo Methods as a designated methodology, during and after his involvement in The Manhattan Project. Famously, Ulam thought up the concept while recovering in a hospital from surgery. He had been playing solitaire to pass the time and wondered whether the probability of success could be estimated statistically. Rather than approaching the problem from a combinatorial approach (which he had already tried and found unsuccessful) he begun to think more 'abstractly' - he had realised that he could simply observe successive games and note outcomes. Crucially for Stanisław Ulam, this could be extended to many simulated trials with the recent advent of computers - which would yield increasingly accurate estimates (Eckhardt 1987).

1.1.3 Subsequent Developments

The link between Ulam's proposed ideas and the following adoption of the statistical approach was a man named John Von Neumann. Perhaps most well-known for his invention of the aptly named Von Neumann architecture (in which programs are stored within a computer much like the data these programs use), he provided the link to the ENIAC - the world's first programmable, electronic digital computer. Von Neumann's computations, refined together with colleagues at Los Alamos laboratory, became the first calculations run on the ENIAC after it had been upgraded to a stored-program paradigm. Nowadays, the computer industry has grown exponentially, and though statistical sampling had already been used in the past, Ulam's thoughts paved the way for the Monte Carlo method to dominate complex statistical problems (Summerscales 2023).

2 Monte Carlo Methods

Having mentioned the basic, essential history, what are Monte Carlo methods, really?

To boil the concept down to its fundamentals, they are simulations and experiments that use randomness in order to solve complex problems. These problems can manifest in different ways, from integrals to systems of equations. Although numerical solutions and standard approximations may yield more exact or precise answers, Monte Carlo methods offer a viable alternative for large dimensionality of integrals and sparse systems of equations, which can often be solved more effectively in terms of cost and time.

2.1 Pseudo-Random Number Generators

Random number generation is an important part of Monte Carlo methods. There are multiple different methods to obtaining random numbers, but the algorithms we are focusing on within this report are called **Pseudo-Random Number Generators** (PRNGs). While they do not generate 'actual' random numbers, what they generate is sufficient enough in variability that it can be used in scenarios where random numbers are needed.

The way that they achieve this is through the use of mathematical formulae, varying in levels of complexity. The algorithms can get increasingly elaborate, yet the simplest can consist of only a few lines of code utilising basic mathematical arithmetic. Sequences are generated which attempt to approximate random numbers, based on an initial starting *seed*, and these sequences will eventually repeat after their *period*. For satisfactory generators, these periods will be sufficiently long enough that the repititions are often unreached by realistic samples (Gentle 2003).

While humans can not predict the next number in the sequences that are generated, this is not perfectly computationally secure. This is because if the initial *seed state* is known, an identical sequence can be reproduced. Therefore, the sequences are deterministic and not as unpredictable as systems with true chaos built into them - such as radioactive atom decay, or, famously, *Cloudflare*'s wall of lava lamps (Liebow-Feeser 2017).

2.1.1 The Linear Congruential Generator (LCG)

One such example of a PRNG, albeit possibly the simplest one, is the Linear Congruential Generator given in the following form:

$$x_i \equiv (ax_{i-1} + c) \bmod m$$

As can be seen, the equation consists of a simple linear function followed by a modular reduction. Although this may not produce sufficiently long streams of random numbers, and the numbers may not appear to be very random themselves, the Linear Congruential Generator and its alternative forms create the basis of many more powerful PRNGs. These alternative forms are built by modifying certain elements, for example setting $c = 0$ (yielding something known as the Multiplicative Congruential Generator)

Here,

- a is called the 'multiplier'
- c is called the 'increment'

- m is called the 'modulus'.

The seed for this generator is the inital value supplied, which is to say x_0. Due to the modular reduction that occurs, there can only be m possible values that x can take; The maximum period of the LCG is, by definition, m (Gentle 2003).

2.1.2 Linear Congruential Generator Example

Following is an example of a LCG displayed in a 'truth table', allowing the process to be broken down into small, simple, and visible steps. Traditionally, values of m are usually a power of 2 as this is easier for the computer to use during calculation. Due to this, I have chosen $m = 2^5 = 32$ which is smaller than a typical usecase. The other values that this LCG will take are $a = 5,\ c = 7,\ X_0 = 19$, which have been arbitrarily selected.

Table 1: **Linear Congruential Generator Example**

aX_n	$+ c$	$(\mathrm{mod}\ m)$	$= X_{n+1}$
$(5 * 19) = 95$	$(95 + 7) = 102$	$102\ (\mathrm{mod}\ 32)$	$= 6$
$(5 * 6) = 30$	$(30 + 7) = 37$	$37\ (\mathrm{mod}\ 32)$	$= 5$
$(5 * 5) = 25$	$(25 + 7) = 32$	$32\ (\mathrm{mod}\ 32)$	$= 0$
$(5 * 0) = 0$	$(0 + 7) = 7$	$7\ (\mathrm{mod}\ 32)$	$= 7$
$(5 * 7) = 35$	$(35 + 7) = 42$	$42\ (\mathrm{mod}\ 32)$	$= 10$
$(5 * 10) = 50$	$(50 + 7) = 57$	$57\ (\mathrm{mod}\ 32)$	$= 25$
$(5 * 25) = 125$	$(125 + 7) = 132$	$132\ (\mathrm{mod}\ 32)$	$= 4$

From the table, we can see that for the Linear Congruential Generator $5X_n+7\ (\mathrm{mod}\ 2^5)$, the starting seed of $X_0 = 19$ provides a sequence of: $6, 5, 0, 7, 10, 25, 4 \cdots$.

Following this, the sequence continues until it starts to repeat, having a period of 32 (the value of m, as stated earlier). The full sequence in its entirety is:

$$19, 6, 5, 0, 7, 10, 25, 4, 27, 14, 13, 8, 15, 18, 1, 12, 3, 22, 21, 16, 23, 26, 9, 20, 11, 30, 29, 24, 31, 2, 17, 28, 19, \ldots$$

2.1.3 The Mersenne Twister

The Mersenne Twister is a more complex, modern PRNG family, developed initially in 1997 (Matsumoto & Nishimura 1998).

It derives its name from a prime number known as a Mersenne Prime, which contributes to its period. As its period is sufficiently large, the algorithm is highly optimised, and the numbers generated have low correlations, the Mersenne Twister is a prominent PRNG that is used across industry. Most notably, it is used in scientific simulations (Monte Carlo methods, as discussed) but also within Machine Learning, statistical sampling and also video game graphics (Samadov 2024).

As, arguably, the most widely used PRNG in the modern day, the Mersenne Twister can be found in many software libraries for standard random generation. These include, but are not limited to, Python, Ruby, and R. One of its multiple variant forms can be used in C++ through `std::mt19937` in the standard library. Due to these occurrences, it is highly likely that within code used in this report (found in the Appendices), the Mersenne Twister is the PRNG that has contributed random elements. For this reason, my C++ simulation (see Appendix C) has employed the use of the mentioned Mersenne Twister.

2.2 Sampling Methodology

By 'Sampling Methodology', we refer to the way in which Monte Carlo methods are used to sample data points (selections to be respresented within subsequent analysis). Depending on the context and scenario, different qualities may be prioritised, and this is reflected in the different ways that sampling can occur (Brownlee 2019).

The 'Law of Large Numbers' plays a significant part in these simulations as that allows the sampling that occurs to form a relvant distribution, despite the initial data being random.

2.2.1 Simple Sampling

Simple sampling, also called a 'crude monte carlo', is the most direct and rudimentary method of using monte carlo sampling techniques. This entails the collection of completely random observations of a random variable. The aforementioned reliance on the law of large numbers leads to a large enough sample size so that an estimate of the expected value ($E[X]$) can be found (*Simple Sampling* 2010).

To see the way that simple sampling works, we can base an example upon the integral of a collected distribution. If a one-dimensional integral I, where $I = \int_a^b \, dx \, f(x)$, were to be re-written as:

$$I = (b-a) \, \langle \, f(x) \, \rangle,$$

this would represent the unweighted average of $f(x)$ over the interval $[a, b]$. As this average is evaluated, larger and larger distributions, tending to infinity, yield a correct value of I (Frenkel & Smit 1996).

An example of this 'crude monte carlo' method can be found in the approximation of Pi mentioned earlier (see Appendices A and B), where uniform and random coordinates are generated within the given square. This results in a calculated average for the value of Pi. This sampling methodology can fall short due to its reliance on randomness; If random values fell unusually within neglibible or heavily influencing areas, the final results of a monte carlo sample can be innacurate - rendering the process inefficient.

2.2.2 Importance

Like Simple sampling, Importance sampling is a method of sampling for data values over an interval, however what sets Importance sampling apart is the addition of weighted 'importance' to certain selected values (hence the name given to the technique). This is achieved through the introduction of a new weight term which we will designate as $p(x)$. This function is commonly known as the *'importance function'*.

Fulfilling a key objective of sampling in the first place, this leads to the variation being reduced to either insignificant levels or, at best, zero in areas where the distribution may not be relevant to draw samples from.

Representing the probability density over a dimension, D, this importance function can be introduced into our earlier estimated integral, making sure $p(x)$ is carefully chosen to minimise variance:

$$\int_D \frac{f(x)}{p(x)} \, p(x) \quad d(x)$$

Variation, with respect to the distribution of X with density $p(x)$, is, hence, given by:

$$V\left(\tfrac{f(X)}{p(X)}\right) \;=\; \left[E\left(\tfrac{f^2(X)}{p^2(X)}\right)\right] \;-\; \left[E\left(\tfrac{f(X)}{p(X)}\right)\right]^2$$

where $\left[E(\frac{f(X)}{p(X)})\right]^2$ is actually equal to $\left(\int_D f(x) \, d(x)\right)^2$, suggesting that only the first expression in the Variance formula has any significant effect on the importance sample.

By Jensen's inequality (see Appendix D for theorem and proof via (Wasserman 2004)), there is a lower bound on this first term. This looks like:

$$\begin{aligned} E\left(\tfrac{f^2(X)}{p^2(X)}\right) \;&\geq\; \left(E\left(\tfrac{|f(X)|}{p(X)}\right)\right)^2 \\ &= \left(\int_D |f(x)| \, d(x)\right)^2 \end{aligned}$$

This is where it is important to select a correct $p(x)$. This bound we have shown is achieved when

$$p(x) = \frac{|f(x)|}{\int_D |f(x)|\, dx},$$

but this is futile when we're considering a monte carlo estimate (and do not know the integral necessarily) - therefore we seek to choose p nearly proportional to $|f|$. Ultimatelty, it is preferable for $\frac{|f(x)|}{p(x)}$ to be nearly constant (Gentle 2003) as this allows variance to, as stated previous, "vanish altogether" (Frenkel & Smit 1996).

2.3 Statistical Mechanics

In order to understand working with particles, and simulations of them, statistical mechanics plays a pivotal part. Within this field, matters of concern include **'macrostates'** and **'microstates'**. Statistical mechanics serves as the bridge connecting microscopic particle behaviour to observable macroscopic properties. A microstate represents a specific, highly detailed configuration of the system at a frozen point in time, such as the exact spatial coordinates of all particles in a liquid simulation. By contrast, a macrostate describes the overall thermodynamic condition of the system, defined by fixed properties (Greiner et al. 1995).

A system at thermal equilibrium would not visit every possible microstate with equal probability. Instead, the probability $P(x)$ of finding the system in a specific microstate x with energy $E(x)$ is proportional to its Boltzmann weight:

$$P(\mathbf{x}) \propto \exp(-\beta E(\mathbf{x}))$$

where $\beta = 1/k_B T$, and k_B is the Boltzmann constant. By generating a sample of microstates that follow this distribution, we can accurately calculate macroscopic thermodynamic averages without needing to evaluate the complex, and often computationally impossible, partition function (Saxena 2016).

2.4 The Metropolis Method

The Metropolis method is an application of importance sampling of system states. Rather than picking completely random configurations, which would likely result in particle overlaps and infinite energies, the algorithm targets configurations with high Boltzmann probabilities.

To ensure the simulation produces a true representation of thermal equilibrium without systematic bias, the transition between states must satisfy the condition of detailed balance. This principle states that the rate of transitioning from an old state i to a new state j must exactly equal the reverse transition rate:

$$P_i W_{i\to j} = P_j W_{j\to i}$$

where P_i is the Boltzmann probability of state i, and $W_{i\to j}$ is the underlying transition probability.

We can split this transition probability into two components: the probability of proposing the move, $q_{i\to j}$, and the probability of accepting it, $A_{i\to j}$. Substituting this into the detailed balance equation gives:

$$P_i q_{i\to j} A_{i\to j} = P_j q_{j\to i} A_{j\to i}$$

By choosing a symmetric proposal distribution—meaning the chance of proposing a random spatial shift from i to j is the same as proposing a shift from j to i ($q_{i\to j} = q_{j\to i}$)—the proposal probabilities cancel out. Rearranging the remaining terms yields the ratio of acceptance probabilities:

$$\frac{A_{i\to j}}{A_{j\to i}} = \frac{P_j}{P_i} = \exp(-\beta(E_j - E_i))$$

To satisfy this ratio, the Metropolis choice for the acceptance probability is defined as:

$$A_{i\to j} = \min\left(1, \exp(-\beta(E_j - E_i))\right)$$

(Frenkel & Smit 1996, Landau & Binder 2021).

3 The Lennard-Jones Potential

3.1 A Description

The Lennard-Jones potential describes the potential energy of two non-bonded particles based on their distance of separation. The equation accounts for both attractive forces and repulsive forces.

The equation is given as follows

$$V(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]$$

where:

- V indicates the intermolecular potential between particles,
- ε is the well depth: the maximum strength of attractive force and minimum energy value
- σ is referred to as the **'van der Waals'** radius, and is a measure of how close two nonbonding particles can get. It measures the distance at which the intermolecular potential between the two particles is zero.
- r indicates distance of separation between both particles, measured from the center of each.

The σ/r^{12} term models strong, short-range repulsion and the σ/r^6 term models long-range van der Waals attraction.

Sometimes the potential can be rearranged and expressed alternatively as

$$V(r) = \frac{A}{r^{12}} - \frac{B}{r^6}$$

in which A, B are equal to $4\varepsilon\sigma^{12}, 4\varepsilon\sigma^6$ respectively (*Lennard-Jones Potential* 2025).

3.1.1 Displaying The Potential Visually

The Lennard-Jones plot can be shown in a graphical manner, representing the various elements of the equation, as described earlier. Setting σ, $\varepsilon = 0$, for simplicities sake, generates the following graph, programmed in python (see Appendix E) using, primarily, the matplotlib library (Hunter 2007).

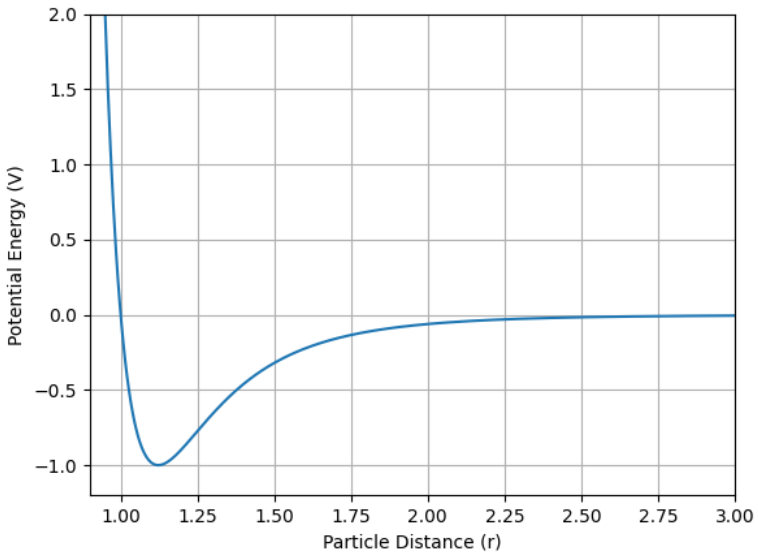


Figure 1: **Lennard-Jones Potential Plot**

In order to facilitate the showcasing of more details on certain parts of the graph, there is also a physical sketch included below. This highlights where certain variables come into play in different parts of the plot, such as σ and ε, alongside the repulsion and attraction aspects.

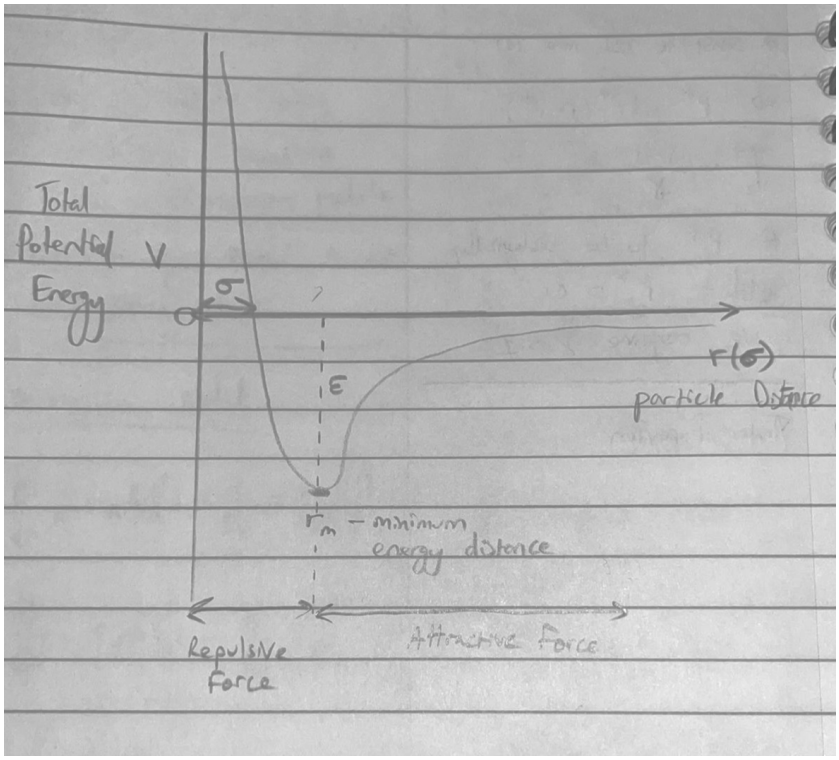


Figure 2: **Lennard-Jones Potential Sketch**

3.1.2 Applications in Modelling Simple Liquids

Hansen and Mcdonald (cited by Ingebrigsten et al.) refer to a **'simple liquid'**, within statistical mechanics, as "a classical system of approximately spherical, nonpolar molecules interacting via pair potentials" (Ingebrigtsen et al. 2012). Because these particles lack directional connections (unlike tangled chains of polymers), their interactions depend purely on the straight-line distance between their centres. What that means is that interactions are fuelled purely by influence from other particles in the same system. This makes the Lennard-Jones model an ideal and computationally efficient mathematical representation of said system as it provides a description of forces acting on these particles.

The Lennard-Jones potential is highly relevant because it captures the two opposing physical forces required for a liquid state to exist:

- **Cohesion (The $(\sigma/r)^6$ term):** This models the long-range van der Waals attractive forces. Without this weak pull, the particles would simply bounce off one another and expand infinitely as an ideal gas. This term holds the particles together to form a condensed liquid phase.
- **Volume (The $(\sigma/r)^{12}$ term):** If particles only attracted one another, the system would eventually collapse into a single dense point. This term models the repulsion that occurs, preventing total collapse and giving the particles a defined physical volume.

By balancing these attractive and repulsive forces, the Lennard-Jones potential would allow a computer simulation to naturally simulate a basic physics system for the particles created.

3.2 Periodic Boundary Conditions

Boundary conditions help to define how a simulation may handle particles within a system. Periodic boundary conditions, in particular, aid in abstracting a small simulation of particles so that it seems like they are interacting within an infinite system. This is done by letting particles cross over the boundaries of the simulation and into identical neighbouring 'cells', i.e cells are tiled infinitely and particles leaving the cell re-enter through the opposite face which can be seen in **Figure 3** (Katiyar & Jha 2018). This allows a much larger system to be simulated with a smaller computational load (as the simulation only really has fewer particles that are replicated and extrapolated).

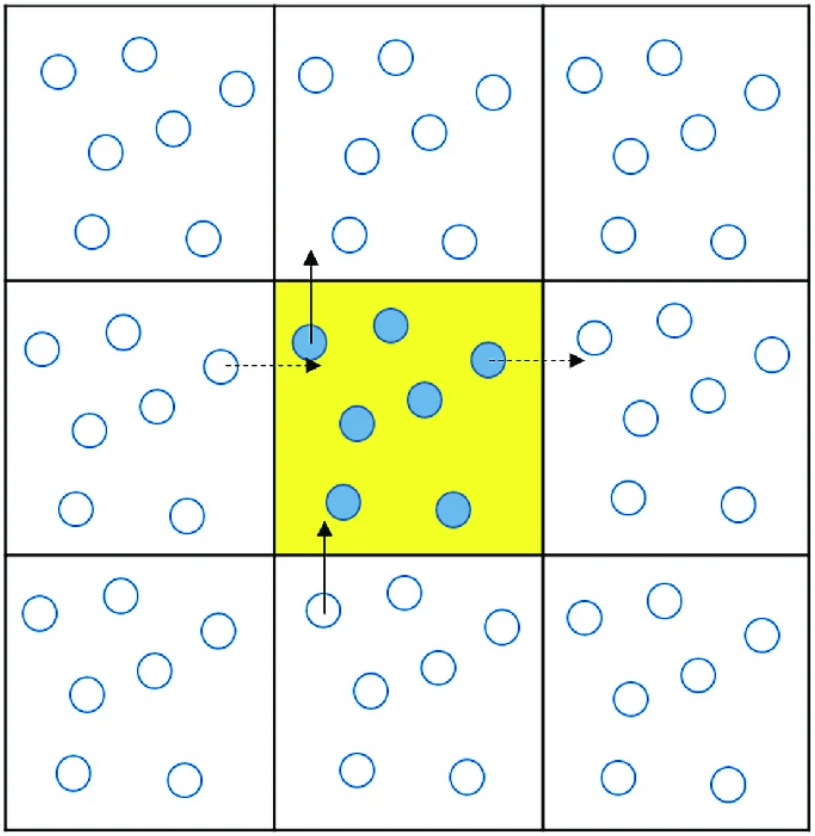


Figure 3: **Periodic Boundary Conditions**

When calculating the interaction energy between a Particle A and Particle B, program code would not just look at a single simple distance. Rather, Particle B and all of its infinite **'periodic images'** (the clones in the adjacent cells) would be considered. The simulation would then calculate the distance to whichever image is physically closest to Particle A. This ensures that particles near the boundary of the primary box correctly interact with the fluid surrounding them.

4 Monte Carlo Simulation: Code And Development

4.1 Steps Toward Implementation

4.1.1 Drawing Together The Theory

In order for correct implementation of the code, all of this theory needs to be drawn together appropriately.

Some of these concepts are built into the software that I am programming with, such as PRNGs, they are very commonly present in random functions that are found in code libraries. As such is the case, I will be using an inbuilt Mersenne Twister within my program code.

The Lennard-Jones potential, previously mentioned, will be used within the code in order for calculations to be taken on particle distances and repulsions. This will be hard-coded into an energy calculation function.

To practically implement Periodic Boundary Conditions, the simulation relies on Minimum Image Convention, which is a rule within simulations stating that only the closest copies of particles should interact with another (even though there are infinitely many cells). To carry this out when computing the distance between two particles, the distance vectors (dx and dy) are adjusted by subtracting the simulation box length multiplied by the rounded ratio of the distance to the box length. This makes sure that each particle interacts only with the closest image of its neighbours, simulating a liquid without having to calculate an impossibly large number of interactions.

4.1.2 Initial Configuration

Before trajectory generation can begin, the simulation environment must be established. The system is initialised with $N = 150$ particles. A reduced density of $\rho^* = 0.8$ is chosen, which dictates how big the simulation's box length should be ($L = \sqrt{N/\rho^*}$).

To prevent particles from spawning on top of one another, which would trigger infinite energy through the $1/r^{12}$ Lennard-Jones term, the particles are initialised onto a regular lattice.

Finally, the Mersenne Twister PRNG is initialised with a fixed seed of 32005096. Although testing and development used a random seed, which has been left within comments in the code, the fixed seed guarantees that random displacements and conditional acceptances can be reproducible for evaluation purposes.

4.1.3 Trajectory Generation Loops

The simulation runs for 5000 Monte Carlo cycles. Within each cycle, the algorithm attempts N individual trial moves to ensure every particle has a statistical chance of displacement.

Regarding the Metropolis method, the derivation found in 2.4 (on page 5) translates into a straightforward algorithmic loop. A particle is randomly selected and given a small trial displacement. The change in energy, $\Delta E = E_j - E_i$, is calculated. If $\Delta E < 0$, the acceptance probability evaluates to 1, and the move is automatically accepted. If $\Delta E > 0$, the move is accepted conditionally by comparing the Boltzmann weight $\exp(-\Delta E/T)$ (using reduced units where $k_B = 1$) against a generated pseudo-random number. This occasional acceptance of uphill energy moves allows the system to escape local energy minima and accurately sample the phase space.

4.2 Post-Simulation Data Analysis

The majority of the post-simulation data analysis is handled in an R program (see Appendix F). During the simulation, a tracking function isolates distinct points: every 10 cycles, the total energy of the system is calculated and recorded to monitor equilibration.

Simultaneously, the distances between all particle pairs are sampled and tallied into histogram bins to track structural data, in order to compute the **'radial distribution function'** later on. The radial distribution function is a tool in statistical mechanics used to describe the internal structure of a fluid. It defines the probability of finding a particle at a specific distance r from a chosen reference particle, relative to the probability expected in a completely uniform, non-interacting ideal gas.

Physically, $g(r)$ reveals the structure of the system. In crystalline solids, particles are locked in fixed lattice positions, resulting in $g(r)$ displaying an infinite series of sharp peaks. In a simple liquid, however, $g(r)$ will display a few distinct peaks at short distances, representing tightly packed shells of nearest-neighbour atoms, before decaying and evaluating out to a value of 1 at larger distances (LibreTexts 2021).

Once the 5000 cycles are complete, this data is exported into **'csv'** files (see Appendices G and H). Externally, programmed in R, the `ggplot2` library is used to plot the data after import. Statistical normalisation takes place in order to convert the raw rdf data into a radial distribution function.

4.3 Code Origin and Inspiration

My code was built on a framework of **Fortran** code from 'Understanding Molecular Simulation' by Daan Frenkel and Berend Smit (Frenkel & Smit 1996). Another Monte Carlo simulation of the Lennard-Jones model (Kelter et al. 2015), packaged within a library known as NetLogo (Wilensky 1999), also provided much guidance and inspiration. These were adapted and rewritten from scratch, by me, into **C++**. My program can be found in the appendix of this report (see Appendix E).

5 Application of Code to a Lennard-Jones liquid

5.1 Total Energy in Plots

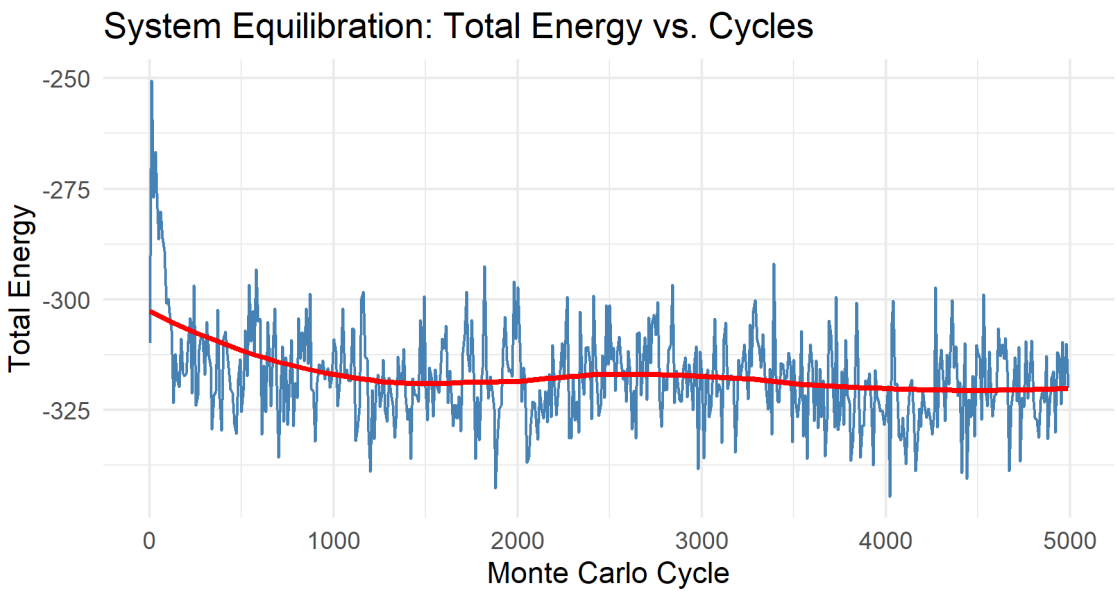


Figure 4: **Total Energy of Simulation**

The plot above displays the total potential energy of the system across the 5000 Monte Carlo cycles. Initially,

the system begins with an artificial configuration because the 150 particles were specifically placed onto a uniform lattice to prevent infinite repulsion and overlap.

As the Metropolis algorithm begins to iterate, the system rapidly explores new microstates, causing fluctuations in the total energy. Around the 1000-cycle mark, the red trendline starts to demonstrate a stabilisation of sorts, with the total energy plateauing and fluctuating steadily around the -320 to -330 reduced energy mark.

This plateau is proof of equilibration. It shows that the starting lattice has completely settled into a liquid and the system has reached a stable equilibrium corresponding to a Lennard-Jones liquid. The continuous fluctuations around this plateau represent the natural thermodynamic variance of a system at equilibrium within an ensemble.

5.2 Radial Distribution Function

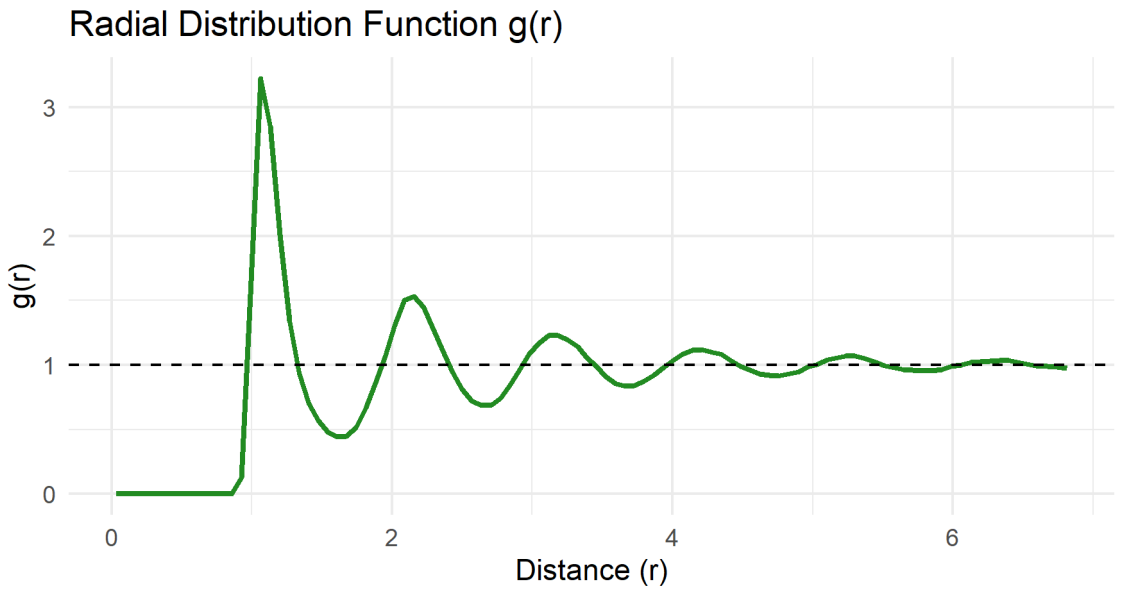


Figure 5: **Radial Distribution Function**

To analyse the structural properties of the simulated liquid, a Radial Distribution Function, $g(r)$, was computed. The raw counts extracted from the C++ code were normalised in R (using the formula $2\pi r \Delta r \rho$). The resulting graph demonstrates the physical characteristics of a simple liquid:

- **The 'Dead Zone' ($r < 0.9$):** For distances less than 0.9, $g(r)$ is exactly zero. This proves the successful implementation of the $1/r^{12}$ repulsion term in the code; atoms cannot physically overlap, so the probability of finding two particles this close together is zero.
- **The First Coordination Shell ($r \approx 1.1$):** The massive spike just after the dead zone represents the first coordination shell. It highlights the high probability of finding a tightly packed ring of nearest-neighbour atoms clustered immediately around any given reference particle.
- **Short-Range Ordering ($1.5 < r < 3.0$):** Following the first peak, the graph shows a distinct valley and a subsequent, smaller second peak. This oscillating structure demonstrates short-range ordering, proving the system is a condensed liquid rather than a completely random, structureless gas.
- **Long-Range Limit ($r \to \infty$):** As distance increases, the graph flattens out and tends towards $g(r) = 1$. At these distances, the central reference particle no longer exerts any significant attractive or repulsive influence. Consequently, the local density simply equals the uniform density of the system, acting like an ideal gas. This lack of long-range order distinguishes the liquid phase from a solid crystal (which would display sharp peaks indefinitely).

6 Conclusions, Findings, Prospectives

This project set out to explore numerical simulation methods, specifically focusing on the application of the Monte Carlo Metropolis algorithm to a particle-based system. By using theories of statistical mechanics, importance sampling, and detailed balance, a C++ program was developed to simulate a simple Lennard-Jones liquid of 150 particles.

The findings were routine and the simulation in question ran as expected. The total energy tracking demonstrated that the algorithm correctly guided the system from an initial lattice into a stable thermodynamic equilibrium. In addition to this, post-simulation data analysis conducted in R yielded a Radial Distribution Function that perfectly aligned with the theoretical structural expectations of a simple liquid.

6.1 Limitations and Prospectives:

While the simulation did successfully model liquid behaviour, it was not without limitations. The system was deliberately scaled to simulate $N = 150$ particles and while this count was sufficient to demonstrate core physics behind the liquid, a system of relatively small size is inherently subject to finite-size anomalies. This can even be in spite of mitigating effects of Periodic Boundary Conditions. Additionally, the simulation was constrained to two dimensions, which restricts direct comparative accuracy to real-world three-dimensional bulk fluids.

Were this done again, expanding the simulation into three dimensions would be a logical next step. Dramatically increasing the particle count to observe true macroscopic thermodynamic limits would also be important.

6.2 Broader Physical Implications:

While the simulated Lennard-Jones liquid is a simplified model, the methodologies validated in this project serve as a foundation for more advanced molecular modelling. The ability to derive properties, such as the radial distribution function, from fundamental pair potentials is a crucial tool in materials science. As outlined in the broader scope of statistical mechanics, scaling these exact Monte Carlo principles to more complex systems allows for much without the need for costly experimentation.

6.3 Personal Reflections

From a personal perspective, this project provided insight into theoretical research and computational physics. Translating literature into C++ required a lot of technique and research, such as statistical mechanics, detailed balance and the Boltzmann distribution.

Strategic decision-making directly lead to the choice of simulate a two-dimensional liquid rather than using a cubic lattice.

7 Bibliography

Brownlee, J. (2019), 'A gentle introduction to monte carlo sampling for probability', https://machinelearningmastery.com/monte-carlo-sampling-for-probability/.

Eckhardt, R. (1987), 'Stan Ulam, John von Neumann, and the Monte Carlo Method', *Los Alamos Science* (15), 131–141. Also: Los Alamos National Laboratory Tech. Report LA-UR-88-9068.
URL: https://permalink.lanl.gov/object/tr?what=info:lanl-repo/lareport/LA-UR-88-9068

Frenkel, D. & Smit, B. (1996), *Understanding Molecular Simulation: From Algorithms to Applications*, Academic Press.

Gentle, J. E. (2003), *Random Number Generation and Monte Carlo Methods*, second edn, Springer.

Greiner, W., Neise, L. & Stöcker, H. (1995), *Thermodynamics and Statistical Mechanics*, Springer. Translated from the German by Dirk Rischke.

Hansen, J. P. & McDonald, J. R. (2005), *Theory of Simple Liquids*, 3rd edn, Academic Press, New York.

Hunter, J. D. (2007), 'Matplotlib: A 2d graphics environment', *Computing in Science & Engineering* **9**(3), 90–95.

Ingebrigtsen, T. S., Schrøder, T. B. & Dyre, J. C. (2012), 'What is a simple liquid?', *Phys. Rev. X* **2**, 011011.
URL: https://link.aps.org/doi/10.1103/PhysRevX.2.011011

Katiyar, R. & Jha, P. (2018), 'Molecular simulations in drug delivery: Opportunities and challenges', *Wiley Interdisciplinary Reviews: Computational Molecular Science* **8**, e1358.

Kelter, J., Luijten, E. & Wilensky, U. (2015), 'Netlogo monte carlo lennard-jones model', http://ccl.northwestern.edu/netlogo/models/MonteCarloLennard-Jones.

Kopka, H. & Daly, P. W. (1999), *A Guide to LaTeX*, third edn, Pearson Education.

Landau, D. P. & Binder, K. (2021), *A Guide to Monte Carlo Simulations in Statistical Physics*, fifth edn, Cambridge University Press.

Lennard-Jones Potential (2025). [Online; accessed 2026-04-22].

LibreTexts, C. (2021), 'Radial Distribution Function', https://chem.libretexts.org/@go/page/294273.

Liebow-Feeser, J. (2017), 'Randomness 101: Lavarand in production', https://blog.cloudflare.com/randomness-101-lavarand-in-production/.

Matsumoto, M. & Nishimura, T. (1998), 'Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator', **8**(1).
URL: https://doi.org/10.1145/272991.272995

Metcalfe, T. (2023), 'What was the manhattan project?', *Scientific American* .

Samadov, I. (2024), 'The mersenne twister algorithm. a deep dive into a pseudo-random number generator', *Medium* .

Saxena, A. K. (2016), *An Introduction to Thermodynamics and Statistical Mechanics*, second edn, Alpha Science International, Limited, 2016.

Sharma, V. (2023), 'Estimating pi using monte carlo methods', https://medium.com/the-modern-scientist/estimating-pi-using-monte-carlo-methods-dbdf26c888d6.

Simple Sampling (2010), Lecture Slides for EG2080 Monte Carlo Methods in Engineering.
URL: https://www.kth.se/social/upload/50bf16b7f276547633d8a01b/Theme%203%20-%20Simple%20Sampling.pdf

Summerscales, O. (2023), 'Hitting the jackpot: The birth of the monte carlo method', https://www.lanl.gov/media/publications/actinide-research-quarterly/1123-hitting-the-jackpot-the-birth-of-the-monte-carlo-method.

Wang, C. (2018), Applications of Monte Carlo Methods in Studying Polymer Dynamics, PhD thesis, University of Reading.

Wasserman, L. A. (2004), *All of Statistics: A Concise Course in Statistical Inference. Springer Texts in Statistics*, Springer.

Wilensky, U. (1999), 'Netlogo', http://ccl.northwestern.edu/netlogo/.

A Approximation of Pi in Python, Using Monte Carlo Techniques

ApproximationOfPi/src/python/SimpleImplementation.py

```
#! python3
"""
Simple implementation of approximation of pi.

Steps:
Generate random x, y inside centre of square of side 2
Repeat steps for a large number of points
Calculate ratio of points
Multiply by 4 (pi / 4)
"""

import numpy as np

def estimatePi(samples):
    insideCircle = 0
    for i in range(samples):
        x = np.random.uniform(-1, 1)
        y = np.random.uniform(-1, 1)

        if (x**2 + y**2) <= 1:
            insideCircle += 1
    ratio = insideCircle / samples
    return (ratio * 4)

if __name__ == "__main__":
    print(f"Estimate with 1 Sample:\n{estimatePi(1)}\n")
    print(f"Estimate with 10 Sample:\n{estimatePi(10)}\n")
    print(f"Estimate with 50 Sample:\n{estimatePi(50)}\n")
    print(f"Estimate with 100 Sample:\n{estimatePi(100)}\n")
    print(f"Estimate with 1000 Sample:\n{estimatePi(1_000)}\n")
    print(f"Estimate with 10,000 Sample:\n{estimatePi(10_000)}\n")
    print(f"Estimate with 1,000,000 Sample:\n{estimatePi(1_000_000)}\n")
```

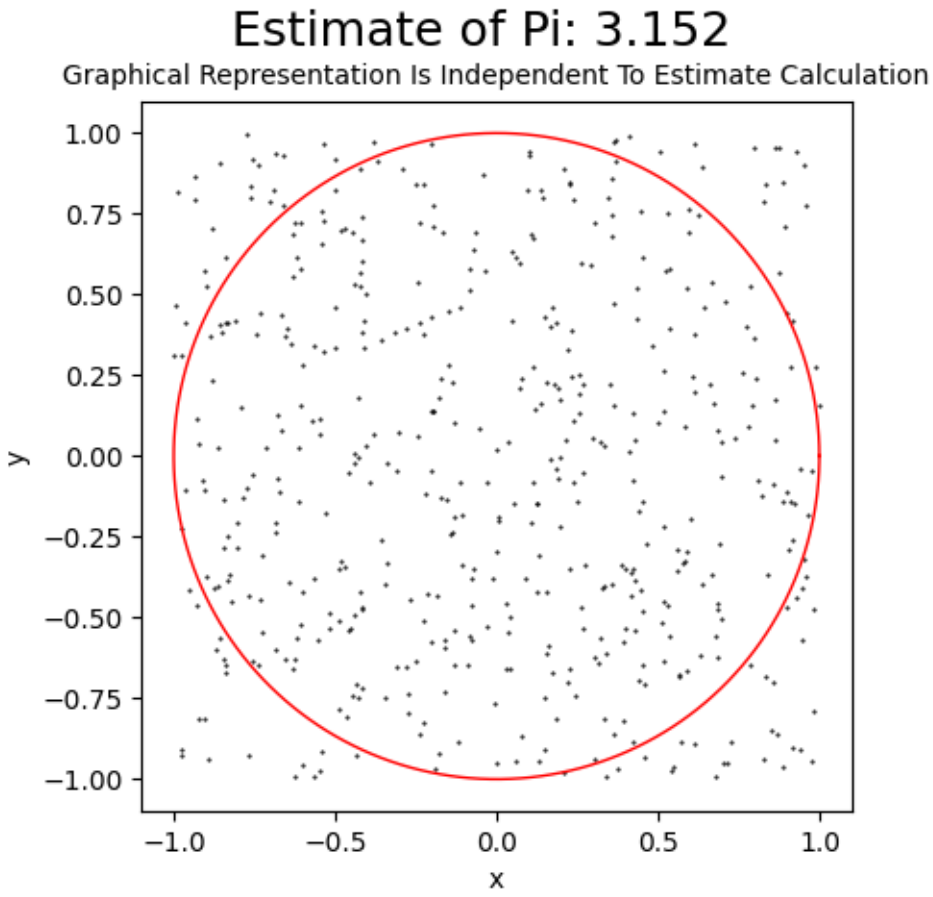
ApproximationOfPi/src/python/GraphicalRepresentation.py

```
#! python3
"""
Graphical representation of approximation of Pi using Monte Carlo methods.
"""
import matplotlib.pyplot as plt
import numpy as np

from SimpleImplementation import estimatePi

def main():
    samples = int(input("Enter Intended Samples:\n"))

    x = np.random.uniform(-1, 1, samples)
    y = np.random.uniform(-1, 1, samples)

    plt.plot(x, y, ".k", markersize = 1)
```

```
    t = np.linspace(0, 2 * np.pi, 400)
    xc = np.cos(t)
    yc = np.sin(t)
    plt.plot(xc, yc, 'r-', linewidth=1)
    plt.gca().set_aspect('equal', adjustable='box')

    plt.xlim(-1.1, 1.1)
    plt.ylim(-1.1, 1.1)
    plt.xlabel("x")
    plt.ylabel("y")

    plt.suptitle(f"Estimate of Pi: {estimatePi(samples)}", fontsize = 18)
    plt.title("Graphical Representation Is Independent To Estimate Calculation",
        fontsize = 10)
    plt.show()

if __name__ == "__main__":
    main()
```

B Approximation of Pi in Python, Code Outputs

```
Estimate with 1 Sample:
4.0

Estimate with 10 Sample:
3.2

Estimate with 50 Sample:
2.96

Estimate with 100 Sample:
3.04

Estimate with 1000 Sample:
3.104

Estimate with 10,000 Sample:
3.1376

Estimate with 1,000,000 Sample:
3.143152
```



C Monte Carlo C++ Code

ParticleSimulation/src/main.cpp

```cpp
#include <iostream>
#include <vector>
#include <cmath>
#include <cstdlib>
#include <string>
#include <random>
#include <fstream>

// To simulate position of particles
struct Position {
    double x, y;
};

const std::string csvTracking_FilePath = "C:\\Users\\rajam\\OneDrive -
    University of Reading\\THIRD YEAR\\PPR\\Numerical-simulation-methods-and-
    their-applications\\ParticleSimulation\\data\\energy_data.csv";
const std::string rdfTracking_FilePath = "C:\\Users\\rajam\\OneDrive -
    University of Reading\\THIRD YEAR\\PPR\\Numerical-simulation-methods-and-
    their-applications\\ParticleSimulation\\data\\rdf_histogram.csv";

// Random Number Generation (Mersenne Twister)
// Using a seed of "32005096" for reproducability

// std::random_device rd; // Random seed used during programming and testing
std::mt19937 gen(32005096);
std::uniform_real_distribution<double> uniform_dist(0.0, 1.0);

int initialisation(int NumParticles, double Density, std::vector<Position>&
    particles, double& box_length) {

    // Calculation of the simulation box length
    // Density (rho) = N / Area. Therefore, Area = N / Density.
    // The length of one side of a square is the square root of its area.
    box_length = std::sqrt(NumParticles / Density);

    // Determining lattice size and spacing
    int n_cells = std::ceil(std::sqrt(NumParticles));
    double spacing = box_length / n_cells;

    // Looping through the grid and place particles
    int p = 0; // Particle counter

    for (int x = 0; x < n_cells; ++x) {
        for (int y = 0; y < n_cells; ++y) {

            // Only place a particle if NumParticles not yet reached
            if (p < NumParticles) {
                Position new_pos;
                new_pos.x = (x + 0.5) * spacing;
                new_pos.y = (y + 0.5) * spacing;

                particles.push_back(new_pos);
                p++;
```

```
            }
        }
    }

    std::cout << "Initialization complete. Placed " << particles.size() << "
        particles.\n";
    return 0;
}

double calc_energy(int idx, const std::vector<Position>& particles, double
    box_length) {
    // Calculation of a Particle's Energy
    double energy = 0.0;
    double rc2 = 6.25; // Cutoff radius squared (2.5 * 2.5)

    for (int i = 0; i < particles.size(); ++i) {
        if (i == idx) continue; // prevent self-interaction of a particle

        // Calculate 2D distance components
        double dx = particles[idx].x - particles[i].x;
        double dy = particles[idx].y - particles[i].y;

        // Periodic Boundary Conditions (Minimum Image Convention)
        dx -= box_length * std::round(dx / box_length);
        dy -= box_length * std::round(dy / box_length);

        double r2 = dx*dx + dy*dy;

        // Lennard-Jones Potential Calculation (only if within cutoff)
        if (r2 < rc2) {
            double r6_inv = 1.0 / (r2 * r2 * r2);
            energy += 4.0 * (r6_inv * r6_inv - r6_inv);
        }
    }
    return energy;
}

// Test initial energy
double calc_total_energy(const std::vector<Position>& particles, double
    box_length) {
    double total_energy = 0.0;
    for (int i = 0; i < particles.size(); ++i) {
        total_energy += calc_energy(i, particles, box_length);
    }
    // Divide by 2 to account for pairs
    return total_energy / 2.0;
}

void mcmove(std::vector<Position>& particles, double box_length, double Temp,
    double max_disp, int& accepted_moves) {

    // Picking a random particle uniformly
    std::uniform_int_distribution<int> particle_dist(0, particles.size() - 1);
    int idx = particle_dist(gen);

    // Calculation of particle energy in current state
    double e_old = calc_energy(idx, particles, box_length);
```

```
    // Saving old position in case of rejection
    Position old_pos = particles[idx];

    // Random displacement proposition
    particles[idx].x += (uniform_dist(gen) - 0.5) * max_disp;
    particles[idx].y += (uniform_dist(gen) - 0.5) * max_disp;

    // PBC
    particles[idx].x -= box_length * std::floor(particles[idx].x / box_length);
    particles[idx].y -= box_length * std::floor(particles[idx].y / box_length);

    // Energy calculation in proposed new state
    double e_new = calc_energy(idx, particles, box_length);
    double delta_e = e_new - e_old;

    // Metropolis Acceptance Criteria
    // If delta_e negative, condition is true, accepted
    // If delta_e positive, Boltzmann probability checked
    if (delta_e < 0.0 || uniform_dist(gen) < std::exp(-delta_e / Temp)) {
        accepted_moves++; // Move accepted, particle stays
    } else {
        particles[idx] = old_pos; // Move rejected, og position restored
    }
}

void simulate_withTracking(int Cycles, int NumParticles, std::vector<Position>&
    particles, double box_length, double Temp, double max_disp, int&
    accepted_moves) {

    std::ofstream outfile(csvTracking_FilePath);
    outfile << "Cycle,TotalEnergy\n";

    // Setup for RDF Histogram variables
    int num_bins = 100;
    double max_r = box_length / 2.0; // Max radius to measure is half the box
    double bin_width = max_r / num_bins;
    std::vector<double> rdf_histogram(num_bins, 0.0);
    int rdf_samples = 0;

    std::cout << "Running " << Cycles << " MC cycles and writing data files...\n
        ";

    for (int c = 0; c < Cycles; ++c) {
        for (int i = 0; i < NumParticles; ++i) {
            mcmove(particles, box_length, Temp, max_disp, accepted_moves);
        }

        // Data Sampling (Every 10 cycles)
        if (c % 10 == 0) {
            double current_energy = calc_total_energy(particles, box_length);
            outfile << c << "," << current_energy << "\n";

            // RDF Sampling
            rdf_samples++;
            for (int i = 0; i < NumParticles - 1; ++i) {
                for (int j = i + 1; j < NumParticles; ++j) {

                    double dx = particles[i].x - particles[j].x;
```

```
                    double dy = particles[i].y - particles[j].y;

                    // PBC for distance measurement
                    dx -= box_length * std::round(dx / box_length);
                    dy -= box_length * std::round(dy / box_length);

                    double r = std::sqrt(dx*dx + dy*dy);

                    if (r < max_r) {
                        int bin = static_cast<int>(r / bin_width);
                        rdf_histogram[bin] += 2.0; // +2 doubled pairs
                    }
                }
            }
        }
    }

    outfile.close();

    // Write to csv
    std::ofstream rdf_file(rdfTracking_FilePath);
    rdf_file << "Distance,Count\n";
    for (int b = 0; b < num_bins; ++b) {
        double r_center = (b + 0.5) * bin_width;
        double average_count = rdf_histogram[b] / rdf_samples;
        rdf_file << r_center << "," << average_count << "\n";
    }
    rdf_file.close();

    std::cout << "Data collection complete. Files closed successfully.\n";
}

int main() {
    // Variables
    double Temp = 1.0;
    int NumParticles = 150;
    double Density = 0.8;
    int Cycles = 5000;
    double max_disp = 0.25; // Maximum allowed distance a particle can jump in
        one step

    std::vector<Position> particles;
    double box_length = 0.0;

    // Initialisation
    initialisation(NumParticles, Density, particles, box_length);
    std::cout << "Initial System Energy: " << calc_total_energy(particles,
        box_length) << "\n\n";

    // Simulation running
    int accepted_moves = 0;
    int total_moves_attempted = NumParticles * Cycles;

    simulate_withTracking(Cycles, NumParticles, particles, box_length, Temp,
        max_disp, accepted_moves);

    // Final Output
    std::cout << "\nFinal System Energy: " << calc_total_energy(particles,
```

```
        box_length) << "\n";
    std::cout << "Acceptance Ratio: " << (double)accepted_moves /
        total_moves_attempted * 100.0 << "%\n";

    return 0;
}
```

D Jensen's Inequality

4.9 Theorem (Jensen's inequality). *If g is convex, then*

$$\mathbb{E}g(X) \geq g(\mathbb{E}X). \tag{4.6}$$

If g is concave, then

$$\mathbb{E}g(X) \leq g(\mathbb{E}X). \tag{4.7}$$

PROOF. Let $L(x) = a + bx$ be a line, tangent to $g(x)$ at the point $\mathbb{E}(X)$. Since g is convex, it lies above the line $L(x)$. So,

$$\mathbb{E}g(X) \geq \mathbb{E}L(X) = \mathbb{E}(a + bX) = a + b\mathbb{E}(X) = L(\mathbb{E}(X)) = g(\mathbb{E}X). \quad \blacksquare$$

From Jensen's inequality we see that $\mathbb{E}(X^2) \geq (\mathbb{E}X)^2$ and if X is positive, then $\mathbb{E}(1/X) \geq 1/\mathbb{E}(X)$. Since log is concave, $\mathbb{E}(\log X) \leq \log \mathbb{E}(X)$.

via (Wasserman 2004)

E Lennard-Jones Potential, Simple Plot in Python

LennardJonesPotential/src/LennardJonesPotential.py

```
#! Python3
"""
Lennard-Jones potential plotted in Matplotlib
"""
import numpy as np
import matplotlib.pyplot as plt
from typing import final


def main():
    r = np.linspace(0, 3.0, 1000)
    SIGMA: final = 1.0
    EPSILON: final = 1.0
    V = 4 * EPSILON * ((SIGMA/r)**12 - (SIGMA/r)**6)

    plt.plot(r, V)

    plt.ylim(-1.2, 2)
    plt.xlim(0.9, 3)
    plt.grid(True)
    plt.ylabel("Potential Energy (V)")
    plt.xlabel("Particle Distance (r)")

    plt.show()

if __name__ == "__main__":
    main()
```

F Data Analysis R Code

ParticleSimulation/src/DataAnalysis.R

```
# Libraries ====
library(ggplot2)

# Setting Up Files ====
assets <- "C://Users/rajam/OneDrive - University of Reading/THIRD YEAR/PPR/
    Numerical-simulation-methods-and-their-applications/assets"

particle.Energy.file <- "ParticleSimulation/data/energy_data.csv"
particle.Energy <- read.csv(particle.Energy.file, header = T)
attach(particle.Energy)

rdf.Data.file <- "ParticleSimulation/data/rdf_histogram.csv"
rdf.Data <- read.csv(rdf.Data.file, header = T)
attach(rdf.Data)

# Total Energy vs. Cycle ====
particle.Energy.plot <- ggplot(
    particle.Energy,
    aes(x = Cycle, y = TotalEnergy)) +
    geom_line(color = "steelblue") +
    geom_smooth(method = "loess",
    color = "red",
    se = F) +
    labs(title = "System Equilibration: Total Energy vs. Cycles",
         x = "Monte Carlo Cycle",
         y = "Total Energy") +
    theme_minimal()

## Saving
assets.Particle <- paste0(assets, "/particle_Energy_plot.png")
ggsave(assets.Particle, plot = particle.Energy.plot, width = 15, height = 8,
    units = "cm")


# Radial Distribution Function ====

## Simulation parameters from main code
density <- 0.8
N <- 150
bin_width <- rdf.Data$Distance[2] - rdf.Data$Distance[1]

# Normalize raw counts to g(r)
rdf.Data$IdealCount <- (2 * pi * rdf.Data$Distance * bin_width) * density * N
rdf.Data$g_r <- rdf.Data$Count / rdf.Data$IdealCount
rdf.Data$g_r[is.nan(rdf.Data$g_r) | is.infinite(rdf.Data$g_r)] <- 0

# Plot Radial Distribution Function ====
rdf.Data.plot <- ggplot(rdf.Data, aes(x = Distance, y = g_r)) +
    geom_line(color = "forestgreen", size = 1) +
    geom_hline(yintercept = 1, linetype = "dashed", color = "black") + # Ideal
        gas limit
    labs(title = "Radial Distribution Function g(r)",
         x = "Distance (r)",
```

```
         y = "g(r)") +
    theme_minimal()

# Save the plot
assets.rdf <- paste0(assets, "/rdf_Data_plot.png")
ggsave(assets.rdf, plot = rdf.Data.plot, width = 15, height = 8, units = "cm")
```

G Particle Energy Data

```
Cycle,TotalEnergy
0,-309.863
10,-250.463
20,-276.888
30,-266.705
40,-275.963
50,-286.343
60,-280.128
70,-286.2
80,-288.988
90,-300.878
100,-299.956
110,-303.117
120,-307.052
130,-323.478
140,-312.36
150,-318.273
160,-319.792
170,-308.859
180,-316.692
190,-317.261
200,-316.745
210,-309.549
220,-304.221
230,-321.301
240,-296.909
250,-324.022
260,-321.785
270,-311.376
280,-307.957
290,-309.199
300,-317.012
310,-305.181
320,-312.712
330,-315.21
340,-329.324
350,-321.849
360,-320.864
370,-302.365
380,-322.465
390,-329.645
400,-308.939
410,-307.363
420,-310.869
430,-315.431
440,-320.306
450,-321.074
460,-328.196
470,-330.333
480,-317.936
490,-313.588
500,-325.466
510,-320.619
520,-307.063
530,-317.328
```

```
540,-296.76
550,-309.517
560,-302.638
570,-312.572
580,-293.174
590,-304.974
600,-304.327
610,-330.468
620,-315.034
630,-325.513
640,-305.149
650,-310.837
660,-324.098
670,-311.052
680,-302.015
690,-319.562
700,-335.832
710,-320.368
720,-310.707
730,-327.621
740,-313.159
750,-328.421
760,-319.265
770,-309.178
780,-328.638
790,-318.87
800,-322.21
810,-304.31
820,-313.333
830,-307.513
840,-313.908
850,-302.142
860,-314.494
870,-298.726
880,-320.295
890,-320.822
900,-332.109
910,-318.716
920,-314.697
930,-316.565
940,-313.842
950,-318.016
960,-316.259
970,-315.558
980,-319.875
990,-317.393
1000,-309.063
1010,-311.472
1020,-324.197
1030,-319.269
1040,-314.596
1050,-302.005
1060,-317.128
1070,-315.833
1080,-318.394
1090,-318.508
1100,-306.596
1110,-306.659
```

```
1120,-331.974
1130,-329.031
1140,-324.626
1150,-299.916
1160,-298.273
1170,-312.958
1180,-313.553
1190,-329.846
1200,-338.984
1210,-320.508
1220,-331.609
1230,-319.349
1240,-317.006
1250,-324.334
1260,-320.881
1270,-313.739
1280,-325.287
1290,-327.587
1300,-317.702
1310,-321.26
1320,-323.917
1330,-331.188
1340,-327.117
1350,-312.976
1360,-318.256
1370,-319.052
1380,-311.245
1390,-322.148
1400,-327.235
1410,-324.689
1420,-336.015
1430,-317.947
1440,-321.573
1450,-321.723
1460,-323.086
1470,-304.707
1480,-319.432
1490,-299.354
1500,-319.962
1510,-327.366
1520,-315.366
1530,-317.398
1540,-326.445
1550,-315.802
1560,-319.74
1570,-321.106
1580,-317.094
1590,-308.931
1600,-311.079
1610,-305.941
1620,-323.453
1630,-316.195
1640,-323.535
1650,-328.689
1660,-323.818
1670,-326.871
1680,-321.967
1690,-329.877
```

```
1700,-312.148
1710,-308.011
1720,-298.255
1730,-312.662
1740,-316.256
1750,-304.723
1760,-316.622
1770,-336.023
1780,-322.12
1790,-331.807
1800,-318.758
1810,-315.368
1820,-292.501
1830,-315.971
1840,-322.34
1850,-317.506
1860,-320.607
1870,-324.687
1880,-342.735
1890,-331.502
1900,-324.776
1910,-324.062
1920,-311.015
1930,-303.993
1940,-313.87
1950,-316.359
1960,-315.929
1970,-317.475
1980,-295.941
1990,-308.923
2000,-297.306
2010,-309.734
2020,-323.162
2030,-315.408
2040,-318.381
2050,-336.949
2060,-335.802
2070,-328.246
2080,-323.083
2090,-323.454
2100,-328.8
2110,-331.694
2120,-320.508
2130,-325.063
2140,-326.895
2150,-320.799
2160,-327.703
2170,-316.911
2180,-326.608
2190,-310.338
2200,-319.836
2210,-326.191
2220,-320.158
2230,-315.967
2240,-309.925
2250,-318.098
2260,-305.041
2270,-299.464
```

```
2280,-331.416
2290,-331.392
2300,-317.243
2310,-327.297
2320,-323.094
2330,-330.132
2340,-302.836
2350,-317.668
2360,-321.584
2370,-310.991
2380,-310.848
2390,-313.787
2400,-327.081
2410,-299.219
2420,-310.098
2430,-318.669
2440,-326.369
2450,-324.172
2460,-312.708
2470,-322.238
2480,-301.415
2490,-314.715
2500,-301.305
2510,-313.547
2520,-310.881
2530,-320.661
2540,-311.7
2550,-308.41
2560,-314.527
2570,-318.09
2580,-316.161
2590,-323.155
2600,-311.717
2610,-316.676
2620,-329.388
2630,-320.669
2640,-331.389
2650,-307.671
2660,-307.439
2670,-321.606
2680,-313.992
2690,-308.618
2700,-322.725
2710,-304.091
2720,-314.403
2730,-305.576
2740,-303.511
2750,-307.99
2760,-300.635
2770,-322.913
2780,-328.805
2790,-316.923
2800,-320.393
2810,-320.515
2820,-311.524
2830,-310.39
2840,-296.771
2850,-323.386
```

```
2860,-316.104
2870,-320.149
2880,-322.688
2890,-323.101
2900,-315.508
2910,-313.108
2920,-322.14
2930,-314.646
2940,-318.73
2950,-325.029
2960,-314.624
2970,-310.574
2980,-338.409
2990,-311.866
3000,-317.351
3010,-335.889
3020,-317.839
3030,-315.905
3040,-326.589
3050,-324.4
3060,-315.97
3070,-304.327
3080,-321.983
3090,-319.24
3100,-320.629
3110,-332.467
3120,-308.932
3130,-305.233
3140,-320.249
3150,-317.65
3160,-317.171
3170,-318.345
3180,-334.619
3190,-327.493
3200,-313.68
3210,-317.33
3220,-314.403
3230,-310.166
3240,-316.053
3250,-322.502
3260,-305.647
3270,-315.102
3280,-302.505
3290,-300.154
3300,-309.191
3310,-310.933
3320,-315.481
3330,-307.926
3340,-317.278
3350,-320.54
3360,-324.861
3370,-315.789
3380,-330.47
3390,-291.9
3400,-317.204
3410,-322.939
3420,-323.211
3430,-314.452
```

```
3440,-314.715
3450,-318.16
3460,-317.704
3470,-312.159
3480,-314.835
3490,-332.331
3500,-313.702
3510,-323.622
3520,-326.591
3530,-322.356
3540,-307.168
3550,-330.914
3560,-321.649
3570,-336.089
3580,-316.466
3590,-310.182
3600,-315.277
3610,-327.437
3620,-313.87
3630,-329.137
3640,-315.723
3650,-326.567
3660,-318.393
3670,-335.543
3680,-327.214
3690,-304.879
3700,-307.926
3710,-319.007
3720,-328.946
3730,-299.473
3740,-329.565
3750,-326.425
3760,-316.243
3770,-328.38
3780,-309.189
3790,-321.845
3800,-323.453
3810,-336.454
3820,-332.319
3830,-322.471
3840,-300.823
3850,-319.282
3860,-335.776
3870,-328.51
3880,-328.618
3890,-318.295
3900,-319.672
3910,-316.68
3920,-324.384
3930,-337.475
3940,-315.955
3950,-322.078
3960,-322.97
3970,-325.329
3980,-325.232
3990,-328.381
4000,-325.541
4010,-319.714
```

```
4020,-344.659
4030,-305.291
4040,-300.314
4050,-319.423
4060,-319.036
4070,-330.795
4080,-331.805
4090,-326.79
4100,-333.333
4110,-337.282
4120,-326.15
4130,-319.593
4140,-318.215
4150,-325.76
4160,-338.8
4170,-332.345
4180,-324.666
4190,-328.845
4200,-320.157
4210,-324.262
4220,-326.772
4230,-324.725
4240,-327.169
4250,-330.597
4260,-325.662
4270,-297.312
4280,-328.975
4290,-315.936
4300,-311.373
4310,-318.131
4320,-327.414
4330,-312.516
4340,-319.251
4350,-311.909
4360,-300.236
4370,-315.466
4380,-310.613
4390,-321.84
4400,-318.761
4410,-339.195
4420,-324.277
4430,-309.889
4440,-340.492
4450,-322.756
4460,-326.658
4470,-317.318
4480,-331.035
4490,-310.402
4500,-312.141
4510,-313.321
4520,-327.241
4530,-298.877
4540,-320.385
4550,-324.146
4560,-311.752
4570,-322.426
4580,-326.462
4590,-321.805
```

```
4600,-321.311
4610,-306.519
4620,-312.769
4630,-310.266
4640,-313.355
4650,-308.741
4660,-316.299
4670,-338.852
4680,-324.007
4690,-320.777
4700,-313.165
4710,-322.947
4720,-310.734
4730,-336.596
4740,-322.296
4750,-324.317
4760,-309.447
4770,-322.352
4780,-316.689
4790,-309.386
4800,-323.088
4810,-326.757
4820,-327.324
4830,-331.247
4840,-322.676
4850,-320.505
4860,-323.335
4870,-312.847
4880,-331.581
4890,-323.574
4900,-317.768
4910,-316.373
4920,-330.117
4930,-311.988
4940,-313.174
4950,-323.659
4960,-309.58
4970,-319.506
4980,-310.046
4990,-320.015
```

H Coordinate Data for Radial Distribution Function

```
Distance,Count
0.0342327,0
0.102698,0
0.171163,0
0.239629,0
0.308094,0
0.376559,0
0.445025,0
0.51349,0
0.581955,0
0.650421,0
0.718886,0
0.787351,0
0.855816,0.008
0.924282,5.976
0.992747,84.584
1.06121,176.592
1.12968,165.996
1.19814,124.368
1.26661,87.8
1.33507,64.512
1.40354,50.896
1.472,42.992
1.54047,37.944
1.60894,36.672
1.6774,38.844
1.74587,46.748
1.81433,62.52
1.8828,83.492
1.95126,107.344
2.01973,136.276
2.08819,161.74
2.15666,170.66
2.22512,166.336
2.29359,151.356
2.36205,135.208
2.43052,118.532
2.49898,104.54
2.56745,95.904
2.63591,93.412
2.70438,96.076
2.77285,106.436
2.84131,124.2
2.90978,144.912
2.97824,167.456
3.04671,184.116
3.11517,197.428
3.18364,201.716
3.2521,200.108
3.32057,196.276
3.38903,185.492
3.4575,176.668
3.52596,166.036
3.59443,158.572
3.66289,158.156
```

```
3.73136,162.052
3.79983,171.988
3.86829,184.216
3.93676,199.156
4.00522,214.02
4.07369,227.796
4.14215,237.984
4.21062,243.44
4.27908,242.64
4.34755,243.044
4.41601,236.32
4.48448,229.684
4.55294,225.824
4.62141,221.632
4.68987,223.368
4.75834,224.688
4.82681,231.296
4.89527,238.796
4.96374,252.164
5.0322,261.592
5.10067,273.54
5.16913,281.856
5.2376,289.836
5.30606,293.288
5.37453,290.768
5.44299,287.42
5.51146,283.296
5.57992,282.252
5.64839,281.572
5.71685,283.084
5.78532,284.688
5.85378,287.992
5.92225,295.82
5.99072,306.308
6.05918,313.044
6.12765,322.408
6.19611,327.804
6.26458,332.32
6.33304,337.744
6.40151,342.128
6.46997,340.144
6.53844,339.692
6.6069,337.132
6.67537,341.34
6.74383,343.184
6.8123,343.836
```